

Lab Assignment 2: Reading and Analysing PGM/PPM Image Headers Using C

Problem Statement:

A digital image file does not contain only pixel values. It also contains information in its header that specifies how the pixel data should be interpreted. In this assignment, work with binary PGM (P5) and/or binary PPM (P6) images and write a C program that reads the image header directly from the file and determines how many bytes of image data must be read.

Tasks:

- **Identify the image format:** Open a PGM/PPM file in binary mode and read its header. Identify and display the magic number (P5 for PGM or P6 for PPM). Explain what the magic number represents.
- **Read the header fields:** Extract the image width, height, and maximum intensity value (MaxVal). Your program must correctly handle whitespace and comments beginning with # in the header.
- **Determine the number of pixels:** Using the width and height obtained from the header, calculate the total number of pixels in the image.
- **Determine bytes per pixel:** For PGM, determine whether each pixel occupies 1 byte or 2 bytes based on MaxVal. For PPM, remember that each pixel contains three colour components (R, G, B), and determine the corresponding number of bytes per pixel.
- **Calculate the expected image-data size:** Using only the header information, calculate the number of bytes that should be read after the header. Display the calculation clearly. For example, distinguish between the cases $\text{MaxVal} \leq 255$ and $\text{MaxVal} > 255$.
- **Read the image data:** After processing the header, read exactly the calculated number of pixel-data bytes from the file. Do not assume a fixed image size. Report the number of bytes successfully read.
- **Verify the file:** Compare the expected pixel-data size with the amount of data actually available in the file. If the values do not match, report that the file may be incomplete, corrupted, or incorrectly interpreted.
- **Analyse the first few pixels:** Display the first 10 pixel values. For PPM, display them as RGB triplets. For a 16-bit image ($\text{MaxVal} > 255$), explain how two bytes represent one sample.
- **Extension versus format:** Test at least one PGM and one PPM file. Check whether the file extension alone is sufficient to identify the image format. Justify your answer using the contents of the file header.
- **Critical-thinking component:** Consider an image with width = 640, height = 480 and MaxVal = 255. Calculate the expected pixel-data size for (i) PGM and (ii) PPM. Then repeat the calculation for MaxVal = 65535. Explain why the number of bytes changes even though the image dimensions remain the same.

Requirement:

The program must be implemented in C using file-handling operations such as `fopen()`, `fgetc()/fscanf()`, `fread()`, and `fseek()/ftell()` where appropriate. Do not use an image-processing library to read the image. The output should clearly show the format, width, height, MaxVal, bytes per sample, bytes per pixel, expected image-data size, actual bytes read, and the first few pixel values.